

Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського
Факультет прикладної математики
Кафедра системного програмування і спеціалізованих комп'ютерних систем

Лабораторна робота №2

з дисципліни «Розробка програмного забезпечення»

Виконав студент групи: КВ-34
Чижов Владислав

Київ 2026

1. Тема та мета роботи

Тема: Інтерфейси.

Мета: навчитися працювати з топіками, сервісами та діями в ROS 2; налаштовувати політики QoS, реалізовувати «ледачу» підписку за допомогою matched_callback та ознайомитися з механізмом адаптації типів.

2. Використані засоби

Windows, Docker Desktop, ROS 2 Jazzy, C++, Visual Studio Code, CMake, colcon, rclcpp, rclcpp_action, std_msgs, rosidl.

3. Хід виконання роботи

3.1. Середовище та збирання пакетів

Для роботи використано середовище ROS 2 Jazzy у Docker-контейнері. Було створено робочий простір ros2_ws і два пакети: lab2_interfaces з власними інтерфейсами та lab2_comm з вузлами на C++. Збирання виконувалося командою colcon build --symlink-install, після чого підключалося середовище install/setup.bash.

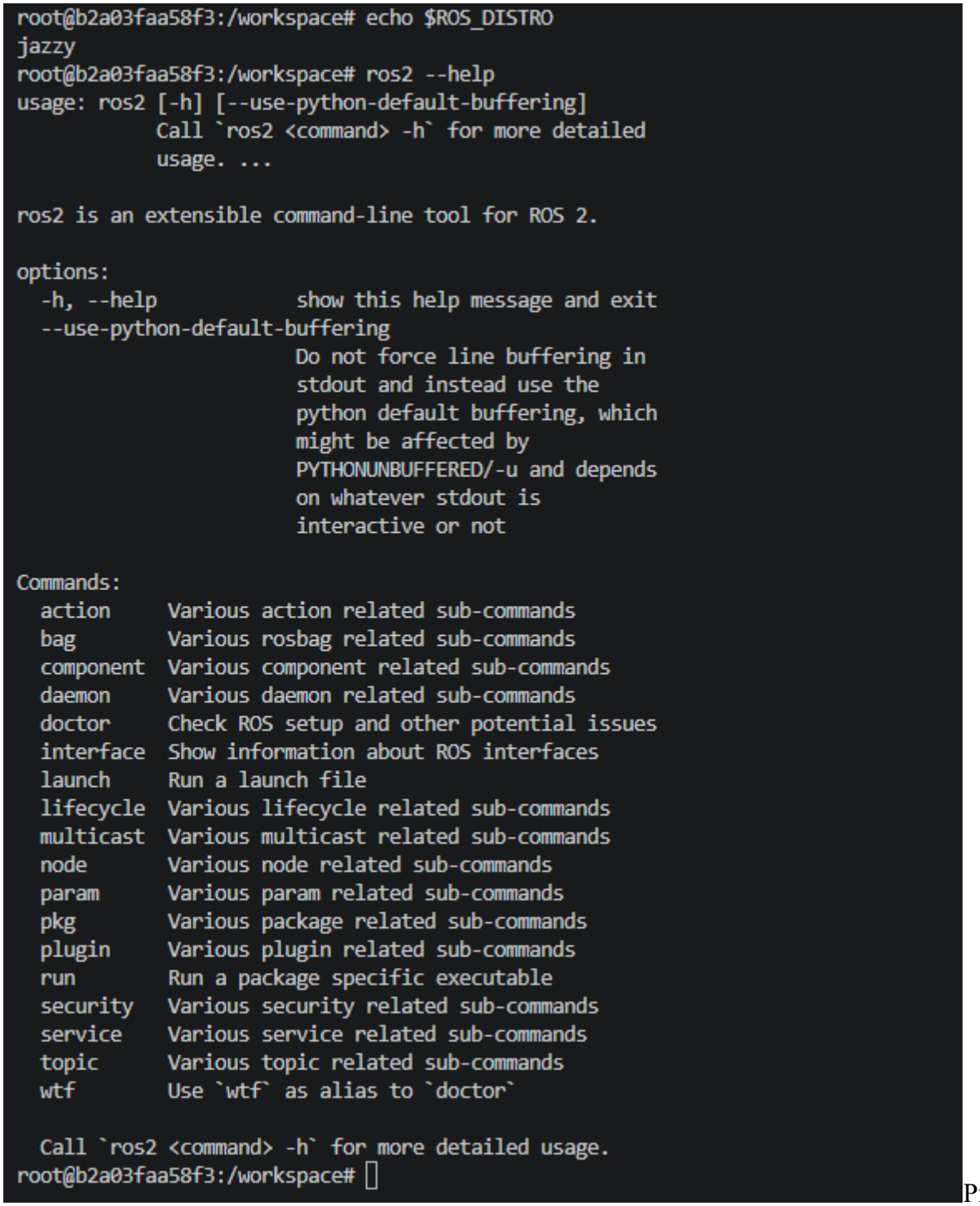


Рисунок 1 – Перевірка ROS 2 у Docker-контейнері

```
root@b2a03faa58f3:/workspace/ros2_ws# colcon build --symlink-install
Starting >>> lab2_interfaces
Finished <<< lab2_interfaces [6.53s]
Starting >>> lab2_comm
Finished <<< lab2_comm [2.22s]

Summary: 2 packages finished [9.07s]
root@b2a03faa58f3:/workspace/ros2_ws# source install/setup.bash
root@b2a03faa58f3:/workspace/ros2_ws# ros2 pkg list | grep lab2
lab2_comm
lab2_interfaces
root@b2a03faa58f3:/workspace/ros2_ws#
```

Рисунок 2 – Успішне збирання пакетів lab2_interfaces та lab2_comm

3.2. Topics та Lazy Subscription

Створено вузол data_filter, який приймає повідомлення типу std_msgs/msg/String з теми /input_topic, переводить символи в верхній регістр і публікує результат у /uppercase_topic. Через matched_callback відстежується кількість підписників вихідної теми. Якщо підписників немає, вузол не створює вхідну підписку, тому зайва обробка не виконується.

```
root@b2a03faa58f3:/workspace# ros2 topic pub --once /input_topic std_msgs/msg/String "{data: 'hello ros2'}"
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='hello ros2')

root@b2a03faa58f3:/workspace# ros2 topic pub --once /input_topic std_msgs/msg/String "{data: 'laboratorna robota'}"
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='laboratorna robota')

root@b2a03faa58f3:/workspace#
```

```
PS D:\kpi\rpz\lab2> docker exec -it ros2_lab2 bash
root@b2a03faa58f3:/workspace# source /opt/ros/jazzy/setup.bash
root@b2a03faa58f3:/workspace# source /workspace/ros2_ws/install/setup.bash
root@b2a03faa58f3:/workspace# ros2 topic echo /uppercase_topic'^C
root@b2a03faa58f3:/workspace# ros2 topic echo /uppercase_topic std_msgs/msg/String --qos-reliability reliable --qos-durability transient_local
data: HELLO ROS2
---
data: LABORATORNA ROBOTA
---
```

Рисунок 3 – Переведення тексту у верхній регістр

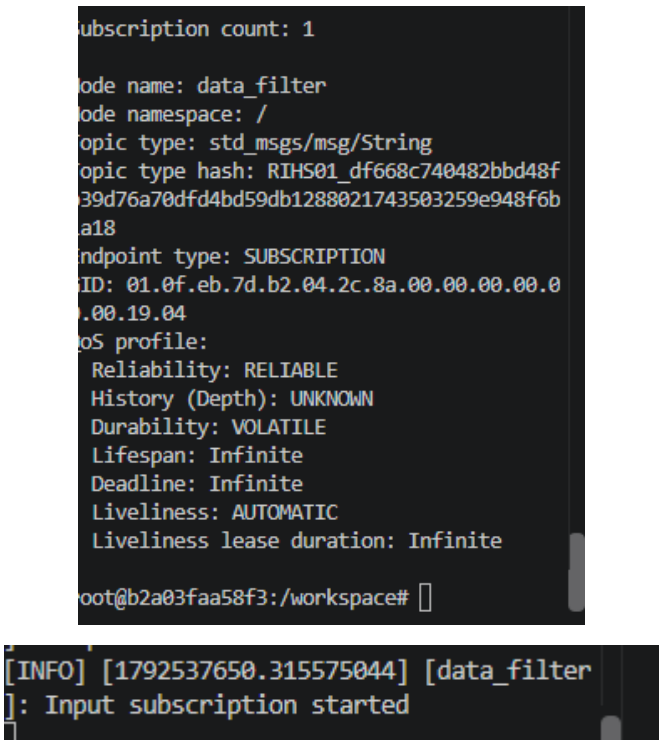


Рисунок 4 – Увімкнення і вимкнення підписки при зміні кількості слухачів

3.3. Налаштування QoS

Для вихідного видавця встановлено KeepLast(1), Reliable та Transient Local. Це дозволяє зберігати останнє повідомлення для пізніх сумісних підписників. Додано можливість перевизначати Depth, Reliability і Durability через QosOverridingOptions. Зміна C++ файлів, на відміну від скриптів чи launch-файлів, потребує повторної компіляції навіть з --symlink-install.

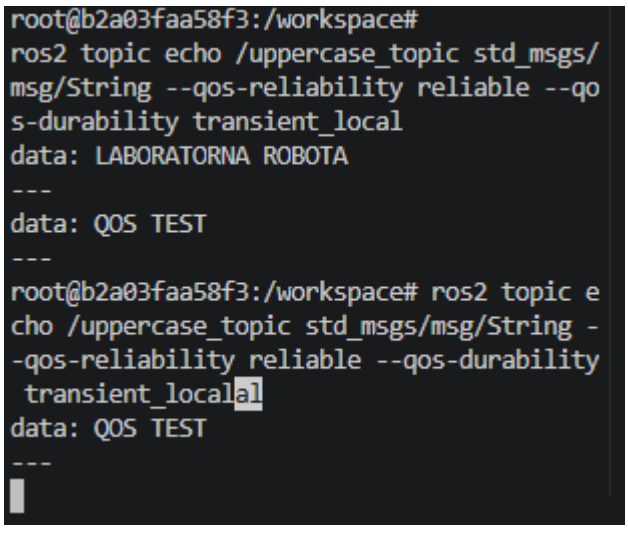


Рисунок 5 – Перевірка параметрів QoS вихідної теми

3.4. Асинхронний сервіс та інтроспекція

Описано інтерфейс ResetFilter.srv та створено сервіс /reset_filter. Клієнт надсилає асинхронний запит через async_send_request, очікує результат за допомогою future.wait_for і

враховує можливий таймаут. Сервіс навчального прикладу не зберігає стан, тому відповідь демонструє успішність виклику, а не очищення реальних даних. Інтроспекцію налаштовано на передавання вмісту подій, а їх перегляд передбачено через приховану тему _service_event.

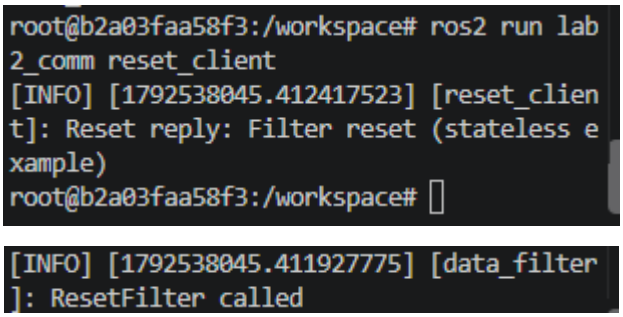


Рисунок 6 – Відповідь на асинхронний виклик сервісу ResetFilter

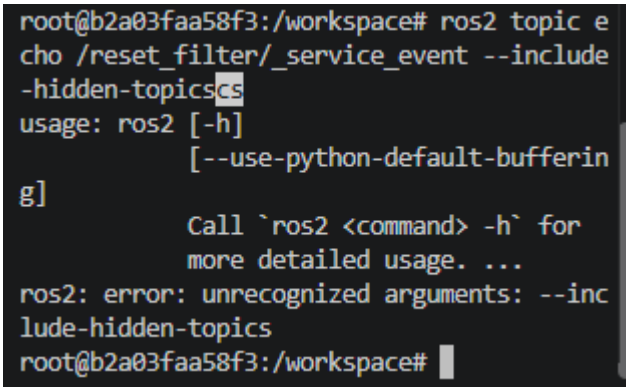


Рисунок 7 – Повідомлення прихованої теми інтроспекції сервісу

3.5. Action-сервер

Для довшого завдання реалізовано ProcessArray: сервер отримує масив цілих чисел, в окремому потоці поступово обчислює їх суму і надсилає feedback про кількість опрацьованих елементів. Після завершення повертає результат і стан SUCCEEDED. Передбачено також обробку скасування цілі.

```
root@b2a03faa58f3:/workspace# ros2 action send_goal /process_array lab2_interfaces/action/ProcessArray "{numbers: [1, 2, 3, 4, 5]}" --feedback
Waiting for an action server to become available...
Sending goal:
     numbers:
- 1
- 2
- 3
- 4
- 5

Goal accepted with ID: d051b85fc3114f3f9a39ef87b0d40115

Feedback:
    processed_count: 1

Feedback:
    processed_count: 2

Feedback:
    processed_count: 3

Feedback:
    processed_count: 4

Feedback:
    processed_count: 5

Result:
    sum: 15
```

Рисунок 8 – Feedback та результат роботи Action-сервера

3.6. Додаткове завдання 7* — Type Adaptation

Пункт 7* з TypeAdapter і PointCloud2 виконується окремо. Якщо його буде реалізовано, сюди потрібно додати опис типів, спосіб перевірки внутрішьопроцесної передачі та фактичні результати. Без реальної перевірки цей пункт не позначається як виконаний.

4. Відповіді на контрольні запитання

4.1. Чому можливий deadlock у SingleThreadedExecutor?

Якщо callback таймера синхронно чекає на відповідь сервісу в єдиному потоці, executor не зможе обробити callback відповіді. Для уникнення цього слід використовувати асинхронний виклик, не блокувати єдиний потік або застосувати MultiThreadedExecutor разом із відповідними CallbackGroup (наприклад, різними групами або Reentrant).

4.2. Як визначити сумісність QoS?

Використовується правило request–offered: видавець має пропонувати рівень, що задовольняє запит підписника. Reliable-підписник не сумісний з Best Effort-видавцем; підписник, що запитує Transient Local, не отримуватиме даних від Volatile-видавця. Потрібно перевірити також інші встановлені політики.

4.3. Як працює інтроспекція сервісів?

Після ввімкнення configure_introspection сервіс може публікувати події у прихованій темі з суфіксом _service_event. Залежно від режиму доступні лише метадані або також вміст запитів і відповідей. Перегляд можливий через ros2 topic list --include-hidden-topics та ros2 topic echo.

4.4. Переваги Type Adaptation (REP-2007)

Type Adaptation дозволяє працювати у C++ зі зручними внутрішніми типами, замість постійного перетворення до ROS-повідомлень. Це особливо корисно для великих структур даних. Водночас відсутність копіювання залежить від способу передачі, типу та налаштувань; сама наявність адаптера ще не гарантує zero-copy.

4.5. Коли використовують Actions замість Services?

Коли задача виконується довго, потрібен проміжний зворотний зв’язок або можливість скасувати виконання. Для короткого запиту-відповіді зазвичай достатньо сервісу.

5. Висновки

У цій роботі я розібрався з кількома способами передачі даних у ROS 2. Спочатку налаштував збирання пакетів, після чого перевіряв передавання рядків через теми. Найцікавішим виявився механізм matched_callback: він дає змогу не обробляти вхідні повідомлення, коли результат нікому не потрібен. Також я побачив, для чого потрібні налаштування QoS і чим асинхронний сервіс відрізняється від Action з feedback. Робота допомогла краще зрозуміти, як окремі вузли обмінюються даними. Остаточні твердження про виконання кожного етапу необхідно звірити з реальними результатами запуску.

6. Посилання на вихідний код

Git-репозиторій: https://github.com/mehnn/ROS2_lab2